CourseLoad

Academic Workload Collision Detector — Software Engineering Project

Project type: Responsive browser-based prototype with a separate C++ console implementation

Student details: Add your name, registration number, class and instructor before submission.

1. Problem Statement

Students manage assignments, quizzes, reports and projects across several courses. When multiple deadlines fall on the same date, the combined workload can be difficult to recognise early. CourseLoad provides a simple place to record tasks and view estimated hours grouped by deadline.

2. Objectives

- Record academic tasks with course, due date and estimated hours.
- Display upcoming tasks and completion status.
- Calculate pending estimated hours for each deadline date.
- Warn when a date exceeds the student's daily study-hour limit.
- Provide a responsive, easy-to-use interface.

3. Scope

The prototype supports adding, viewing, editing, deleting and completing tasks. It groups pending task hours by due date and compares the total with a user-selected daily limit. The website uses browser localStorage. The C++ console version is included as a separate alternative implementation.

4. Stakeholders

Primary stakeholder: university students who need to organise academic deadlines. Secondary stakeholder: the instructor evaluating the problem analysis, software process, prototype and testing.

5. Requirements

Functional: add/edit/delete tasks; view/filter tasks; mark tasks complete; total pending hours by due date; compare with daily limit; save and reload data in the same browser.

Non-functional: clear interface; responsive desktop/mobile layout; immediate interaction; no account or server required for the local prototype.

6. SDLC Model — Incremental

An incremental approach is suitable because the system can be built in small working parts: task entry, task listing, completion controls, workload calculation, and interface refinement. Each increment adds a testable capability to the prototype.

7. Design

The website is divided into three front-end files: index.html defines the page structure, style.css controls layout and appearance, and app.js handles task operations, calculations and browser storage. The diagrams folder contains editable Mermaid source for the use-case-style view, activity flow, system flowchart and conceptual ER model.

8. Data and Constraints

Tasks are stored in localStorage in the browser. Data is limited to that browser and device; it is not synchronised or backed up remotely. The website is not connected to the separate C++ program. The prototype does not provide login, notifications or a detailed study timetable.

9. Testing

Manual test cases are provided in Test_Cases.txt. They cover task creation, persistence after refresh, completion, overload warnings, editing/deleting and mobile layout. Execute each test and record the actual outcome before submission; no test should be reported as passed without running it.

10. Risks and Mitigations

Risk: browser storage may be cleared — mitigation: explain local-only storage and keep backups of important data. Risk: incorrect hour estimates — mitigation: allow students to edit estimates. Risk: mobile layout issues — mitigation: test at phone-sized widths. Risk: scope becoming too large — mitigation: keep the prototype focused on deadlines and daily workload.

11. Future Enhancements

Possible future improvements include cloud synchronisation, login, reminders, calendar integration, recurring tasks and a weekly study timetable. These are outside the current prototype scope.

Submission reminder

The assignment brief specifies groups of three. Obtain instructor approval before submitting individually. Replace generic student details and personally review the diagrams, code and test results so you can explain them during the presentation.